Introduction

In concurrent programming, one of the most classic problems used to explain synchronization is the **Producer-Consumer Problem**. It represents a scenario where two types of threads — producers and consumers — share a common, finite-size buffer. The producer's job is to generate data and place it into the buffer, while the consumer's job is to remove data from the buffer for processing.

This problem helps illustrate the challenges of coordinating threads, avoiding race conditions, and ensuring thread-safe communication between processes.

The Producer-Consumer Problem

The **Producer-Consumer Problem**, also known as the **Bounded Buffer Problem**, is a classic example of a **multi-process synchronization** issue. It revolves around coordinating two types of threads:

- **Producers:** which generate data and add it to a shared buffer.
- **Consumers:** which take data from the buffer and process it.

Challenge

The core challenge lies in ensuring that producers don't add data when the buffer is full, and consumers don't try to remove data when the buffer is empty — all while avoiding race conditions and ensuring mutual exclusion and proper coordination between threads.

This problem is a **synchronization construct** that ensures:

- **Mutual Exclusion:** Only one thread can access the shared buffer (critical section) at a time.
- **Wait-Notify Coordination:** Producers wait when the buffer is full; consumers wait when it's empty. Threads notify each other when buffer states change.

Real-Life Analogies

- **Washroom with One Key:** Think of a washroom with a single key. Only one person can enter at a time (mutual exclusion). If it’s occupied, others must wait outside (wait-notify mechanism).
- **Coffee Machine:** A **coffee machine** acts as a producer that brews coffee. The **customer** is the consumer who drinks it. The machine shouldn't brew more if the cup is already full, and the customer must wait if there's no coffee — a perfect reflection of the coordination needed.

Solution

In this **Producer-Consumer Problem**, the major challenge is coordination — ensuring the producer doesn't add items to a full buffer and the consumer doesn't try to consume from an empty one. This is where Java's `wait()` and `notify()` methods come into play.

`wait()` and `notify()` methods

These two methods provide a **built-in signaling mechanism** to safely pause and resume threads based on the shared resource's state:

- **wait():** Causes the current thread to pause **and release the lock** until another thread signals it using **notify()**.
- **notify():** Wakes up one waiting thread so it can try to re-acquire the lock and proceed.

Together, they eliminate the need for **busy waiting**, reduce CPU usage, and help implement the **wait-notify coordination** pattern — a fundamental synchronization requirement in the producer-consumer scenario.

Example Code

Let's now walk through a simple example where the buffer holds just one item: a cup of coffee.

```
import java.util.LinkedList;

import java.util.Queue;


// Shared resource: CoffeeMachine acts as a buffer of size 1

class CoffeeMachine {

  // Flag to indicate buffer status: true → coffee ready, false → cup empty

  private boolean isCoffeeReady = false;


  // Method to be run by the producer thread

  public synchronized void makeCoffee() throws InterruptedException {

    // If coffee is already ready, producer must wait (buffer is full)

    while (isCoffeeReady) {

      // Releases lock and waits until notified by consumer

      wait();

    }
```

```
    // Simulate coffee preparation
    System.out.println("Brewing coffee...");
    Thread.sleep(1000); // Simulate time to make coffee

    // Set the buffer to full (coffee is now ready)
    isCoffeeReady = true;
    System.out.println("Coffee ready!");

    // Notify the consumer thread that coffee is available
    notify();
}

// Method to be run by the consumer thread
public synchronized void drinkCoffee() throws InterruptedException {
    // If no coffee is available, consumer must wait (buffer is empty)
    while (!isCoffeeReady) {
        // Releases lock and waits until notified by producer
        wait();
    }

    // Simulate coffee consumption
```

```
        System.out.println("Drinking coffee...");
        Thread.sleep(1000); // Simulate time to drink coffee

        // Set the buffer to empty (coffee has been consumed)
        isCoffeeReady = false;
        System.out.println("Cup emptied - brew next!");

        // Notify the producer thread that it can brew again
        notify();
    }
}

// Driver class
class ProducerConsumerProblem {
    public static void main(String[] args) {
        // Shared CoffeeMachine object acts as the synchronized monitor
        CoffeeMachine machine = new CoffeeMachine();

        // Producer thread that continuously makes coffee
        Thread producer = new Thread(() -> {
            while (true) {
```

```
        try {
            machine.makeCoffee(); // Try to produce coffee
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
});

// Consumer thread that continuously drinks coffee
Thread consumer = new Thread(() -> {
    while (true) {
        try {
            machine.drinkCoffee(); // Try to consume coffee
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
});

// Start both threads
producer.start();
```

```
        consumer.start();
    }
}
```

How `wait()` and `notify()` Work Together?

- **Initial State:**
  - The buffer (`isCoffeeReady`) is **false** — no coffee yet.
  - The **producer thread** enters `makeCoffee()` and proceeds to brew coffee.
  - The **consumer thread** enters `drinkCoffee()` but sees no coffee, so it goes to `wait()` and sleeps
- **Producer Brews Coffee:**
  - After brewing (`Thread.sleep(1000)`), it sets `isCoffeeReady = true`.
  - It then calls `notify()` to **wake up the waiting consumer**.
  - The producer thread loops back but now finds coffee still present, so it goes into `wait()`.
- **Consumer Wakes Up and Drinks Coffee:**
  - The consumer, upon waking, sees that the coffee is ready.
  - It drinks the coffee and sets `isCoffeeReady = false`.
  - It then calls `notify()` to **wake up the producer**.
- **Cycle Repeats:**
  - The producer wakes up and sees that coffee has been consumed.
  - It brews another cup, and the entire cycle continues.

Both methods are `synchronized`, so only one thread can run inside them at any time (mutual exclusion).

Key Methods Explained

Method	Purpose	What Actually Happens
`wait()`	Voluntarily releases the intrinsic lock and suspends the current thread	The thread goes into the **WAITING** state. It pauses execution and releases the object's monitor. It will resume only when another thread calls `notify()` or `notifyAll()` on the same object.
`notify()`	Wakes one thread that is waiting on the same object's monitor	A random waiting thread moves from **WAITING** to **BLOCKED** state. It competes to reacquire the object's lock before it can continue executing.

Note that the `while loop` in the `makeCoffee()` and `drinkCoffee()` methods protects against spurious wake-ups and re-checks condition after every resume.

Key Takeaways

- **Single-slot buffer:** Setting `isCoffeeReady` acts as the entire queue; the flag prevents over-production or over-consumption.
- **Intrinsic lock:** Declaring both methods `synchronized` gives us mutual exclusion for free.

- **Condition signalling:** `wait()` + `notify()` form a lightweight, built-in condition-variable mechanism; they let threads sleep without busy-waiting.
- **Always loop around** `wait()` to guard against spurious wake-ups and to re-validate the buffer's state.
- **Scalable pattern:** Replace the boolean with a `Queue<T>` and size checks to move from "tiny" to "bounded-buffer" implementations.

Handling Fast Producer and Slow Consumer (TUF+ Judge Example)

Previously, we studied a solution with a **one-slot buffer**, where the producer (coffee machine) and the consumer (customer) worked in perfect alternation — one producing, the other consuming. But real-world systems often face **imbalance in processing speeds**.

Imagine a scenario in **TUF+**, where the **submission judge** (consumer) takes longer to process each code submission than the time it takes for users (producers) to submit them. If we still used a single-slot buffer, submissions would frequently get blocked or lost due to the judge being busy.

The Problem

In this case:

- **Producers (users)** submit solutions rapidly.
- **Consumer (judge system)** takes time to compile, run, and verify each solution.
- If the buffer can hold only one submission, the producer has to wait too frequently — leading to poor performance and wasted CPU cycles.

The Solution

To fix this, we use a **buffered queue** (like `Queue<String> submissions = new LinkedList<>();`) that can store **multiple submissions at once**. This allows:

- **Producers to keep submitting** even when the consumer is busy.
- **Consumers to process submissions at their own pace**, pulling from the buffer.

This change models how real-world systems like TUF+ handle bursty traffic and slower backend processing.

Code

```
import java.util.LinkedList;

import java.util.Queue;


// Represents a code submission by a user

class Submission {

   private static int idCounter = 1; // Used to generate unique submission IDs

   private final int submissionId;

   private final String userName;


   public Submission(String userName) {

      this.userName = userName;

      this.submissionId = idCounter++; // Auto-incrementing ID
```

```
    }

    public int getSubmissionId() {
        return submissionId;
    }

    public String getUserName() {
        return userName;
    }
}

// Shared resource between producers (users) and consumers (judges)
class SubmissionQueue {
    private final Queue<Submission> queue = new LinkedList<>(); // Shared buffer
    private final int MAX_CAPACITY = 5; // Bounded buffer size

    // Producer logic: User submits a solution
    public synchronized void submit(Submission submission) throws InterruptedException {
        // If queue is full, producer waits
        while (queue.size() == MAX_CAPACITY) {
```

```
            System.out.println("⏳ Queue full. " + submission.getUserName() + "
is waiting to submit.");
            wait(); // Releases lock and waits for space
        }

        // Add submission to the queue
        queue.offer(submission);
        System.out.println("" + submission.getUserName() + " submitted
code: #" + submission.getSubmissionId());

        notifyAll(); // Notifies judges that a new task is available
    }

    // Consumer logic: Judge processes a submission
    public synchronized Submission consume(String judgeName) throws
InterruptedException {
        // If queue is empty, consumer waits
        while (queue.isEmpty()) {
            System.out.println("△ " + judgeName + " waiting for submissions...");
            wait(); // Releases lock and waits for submissions
        }
```

```
        // Remove a submission from the queue
        Submission sub = queue.poll();
        System.out.println("⚙️ " + judgeName + " started evaluating submission #" +
                           sub.getSubmissionId() + " from " + sub.getUserName());

        notifyAll(); // Notifies waiting producers if queue was full
        return sub;
    }
}
```

Understanding Key Methods

Method	Purpose	Behavior
`wait()`	Pause the current thread and release the lock	Thread enters the **WAITING** state and resumes only when another thread calls `notify()` / `notifyAll()` on the same object
`notifyAll()`	Wake all threads waiting on the object's monitor	All waiting threads become **BLOCKED**; the first to reacquire the lock continues execution

`queue.offer()`	Add a submission to the buffer (if space exists)	Called by producers to enqueue a new task
`queue.poll()`	Remove a submission from the front of the buffer	Called by consumers to dequeue and process a task

Key Takeaways

- **Buffered Queue:** We now use a queue of size 5 to hold multiple submissions — unlike the previous one-slot version.
- **Fast Producers, Slow Consumers:** Producers don't have to wait unless the buffer is full. These model systems like TUF+ where many users submit rapidly, but judges can evaluate only one at a time.
- `notifyAll()` **vs** `notify()`**:** `notifyAll()` is used because multiple threads (many users or multiple judges) might be waiting — all should get a chance.
- **Thread Coordination:** Both producer and consumer use `wait()`-`notifyAll()` mechanism to signal when the buffer is full or empty.
- **Thread Safety:** All shared state is accessed inside `synchronized` methods to maintain consistency.

Conclusion

The Producer-Consumer problem is a foundational concept in multithreading that teaches us how to manage coordination between threads effectively. By moving from a one-slot buffer to a bounded queue,

we modeled a real-world scenario like the TUF+ judge system, where producers can be faster than consumers. Using `wait()` and `notifyAll()` ensures smooth synchronization, prevents data loss, and maintains thread safety — all essential for building reliable concurrent applications.